

## ✓ CS4085 Deep Learning — Lab 2

### Planar Data Classification with One Hidden Layer

**Effat University · College of Engineering · Computer Science Instructor:** Dr. Naila Marir

In this lab you will build your first neural network: a 2-class classifier with **one hidden layer**. You will see how much better it does than logistic regression on data that is not linearly separable.

#### Learning outcomes

By the end of this lab you will be able to:

1. Implement a 2-class classification neural network with a single hidden layer.
2. Use units with a non-linear activation function, such as `tanh`.
3. Compute the cross-entropy loss.
4. Implement forward and backward propagation, and train the network with gradient descent.
5. Explain how the hidden-layer size and the learning rate change what the network learns.

#### Table of Contents

- [1 - Packages and lab utilities](#)
- [2 - Load the Dataset](#) — Exercise 1
- [3 - Simple Logistic Regression](#)
- [4 - Neural Network model](#)
  - 4.1 Defining the neural network structure — Exercise 2 `layer_sizes`
  - 4.2 Initialize the model's parameters — Exercise 3 `initialize_parameters`
  - 4.3 Forward propagation — Exercise 4 `forward_propagation`
  - 4.4 Compute the cost — Exercise 5 `compute_cost`
  - 4.5 Backpropagation — Exercise 6 `backward_propagation`
  - 4.6 Update parameters — Exercise 7 `update_parameters`
  - 4.7 Integration — Exercise 8 `nn_model`
- [5 - Test the Model](#) — Exercise 9 `predict`
- [6 - Tuning the hidden layer size](#)
- [7 - The learning rate](#)
- [8 - Performance on other datasets](#)
- [9 - Reflection questions](#)

## 1 - Packages and lab utilities

- [numpy](#) is the main package for scientific computing in Python.
- [scikit-learn](#) gives simple tools for data mining and data analysis.
- [matplotlib](#) is a library for plotting graphs in Python.

All of these are already installed in Google Colab, so there is nothing to install.

```
# Package imports
import numpy as np
import copy
import matplotlib.pyplot as plt
import sklearn
import sklearn.datasets
import sklearn.linear_model

import warnings
warnings.filterwarnings("ignore", category=FutureWarning) # hide scikit-learn version no

%matplotlib inline
```

The next cell contains everything the lab needs: the dataset loaders, `sigmoid()`, a plotting helper for decision boundaries, and **self-check tests** for each exercise. In Colab it appears collapsed.

**Run it, but do not edit it.**

### > Lab 2 utilities — run this cell, do not modify

[Show code](#)

Lab 2 utilities loaded.

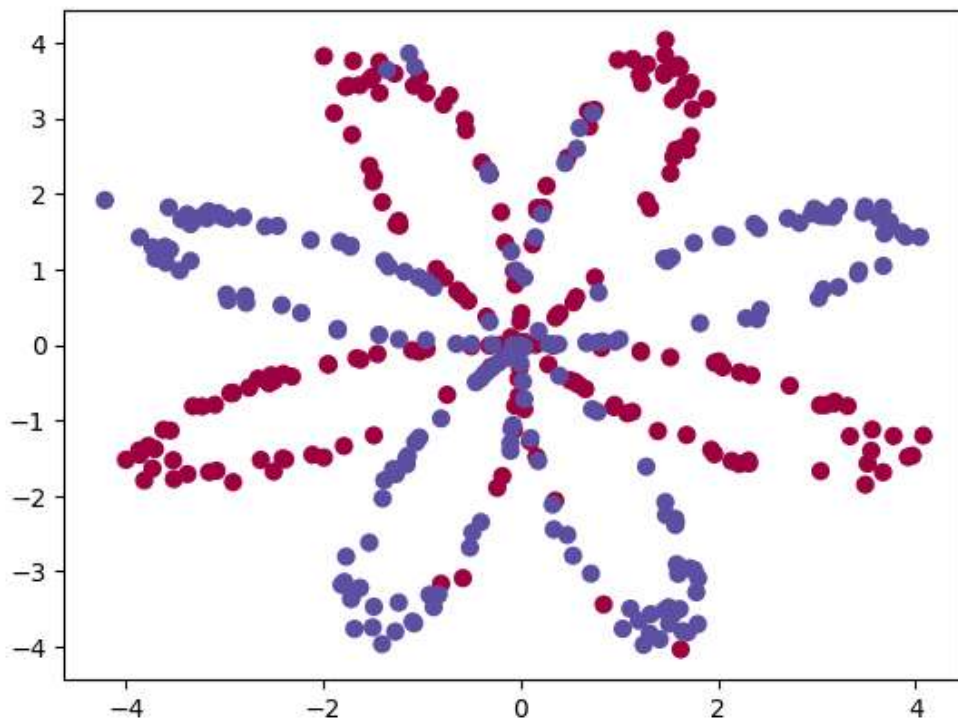
## 2 - Load the Dataset

The code below loads a 2-class dataset into the variables `X` and `Y`.

```
X, Y = load_planar_dataset()
```

Visualize the dataset using matplotlib. The data looks like a "flower" with some red (label  $y=0$ ) and some blue ( $y=1$ ) points. Your goal is to build a model that fits this data: the classifier should label every region of the plane as either red or blue.

```
# Visualize the data
plt.scatter(X[0, :], X[1, :], c=Y.ravel(), s=40, cmap=plt.cm.Spectral);
```



You have:

- a numpy array (matrix) `X` that contains your features ( $x_1, x_2$ )
- a numpy array (vector) `Y` that contains your labels (red: 0, blue: 1).

First, get a better sense of what your data looks like.

## ✓ Exercise 1

How many training examples do you have? What is the `shape` of the variables `X` and `Y`?

**Hint:** How do you get the shape of a numpy array? ([help](#))

```
shape_X = np.shape(X)
shape_Y = np.shape(Y)
m = X.shape[1]
# training set size

print('The shape of X is: ' + str(shape_X))
print('The shape of Y is: ' + str(shape_Y))
print('I have m = ' + str(m) + ' training examples!')

dataset_shape_test(shape_X, shape_Y, m)
```

```
The shape of X is: (2, 400)
The shape of Y is: (1, 400)
I have m = 400 training examples!
All tests passed for Exercise 1!
```

## Expected output

```
The shape of X is: (2, 400)
The shape of Y is: (1, 400)
I have m = 400 training examples!
```

### ✓ 3 - Simple Logistic Regression

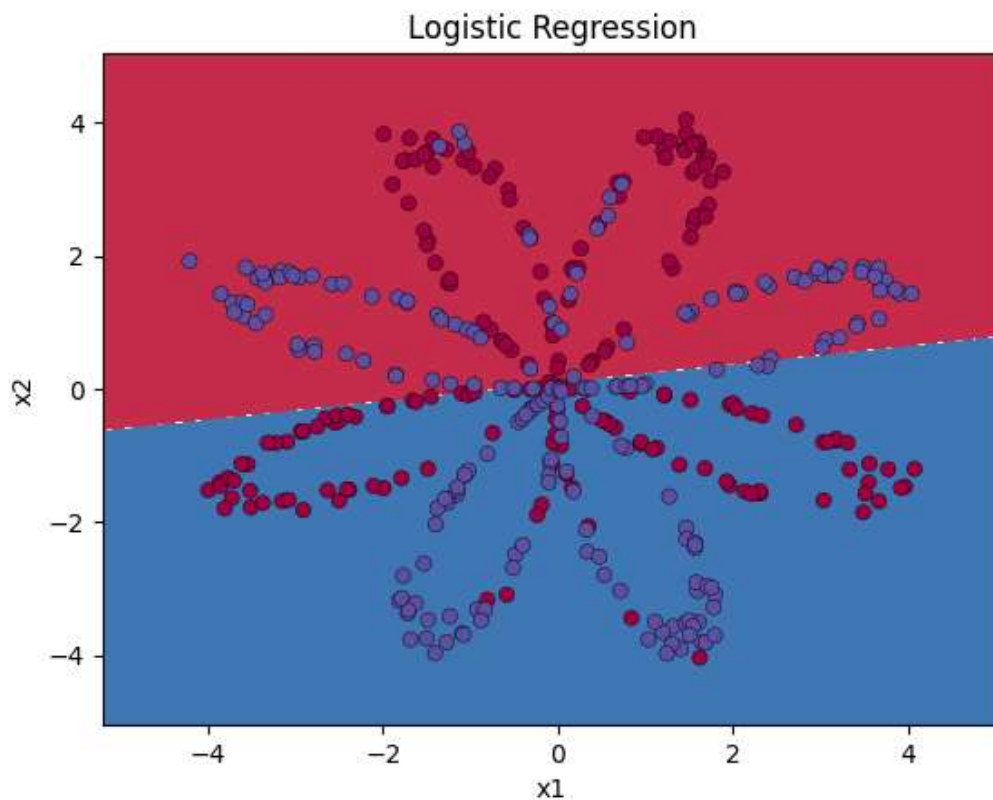
Before you build a full neural network, check how logistic regression does on this problem. Run the code below to train scikit-learn's logistic regression classifier on the dataset.

```
# Train the logistic regression classifier
clf = sklearn.linear_model.LogisticRegressionCV()
clf.fit(X.T, Y.ravel());
```

Now plot the decision boundary of this model. Run the code below.

```
# Plot the decision boundary for logistic regression
plot_decision_boundary(lambda x: clf.predict(x), X, Y)
plt.title("Logistic Regression")
plt.show()

# Print accuracy
LR_predictions = clf.predict(X.T)
LR_accuracy = np.mean(LR_predictions == Y.ravel()) * 100
print('Accuracy of logistic regression: %d%% (percentage of correctly labelled datapoints)
```



Accuracy of logistic regression: 47% (percentage of correctly labelled datapoints)

**Expected output:** Accuracy of logistic regression: 47%

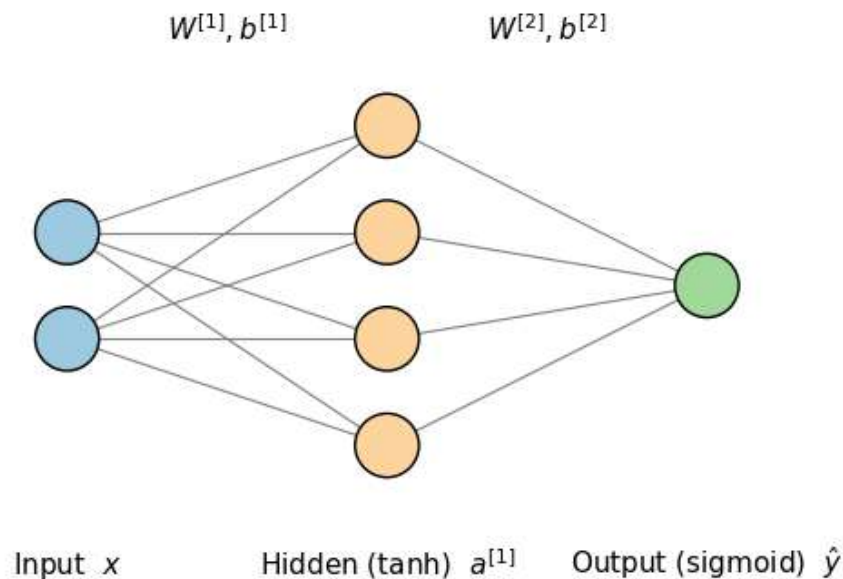
**Interpretation:** The dataset is not linearly separable, so logistic regression does not do well. A neural network should do better. Let's try it now!

## 4 - Neural Network model

Logistic regression did not work well on the flower dataset. Next, you will train a neural network with a single hidden layer and see how it handles the same problem.

**The model** (run the cell below to draw it):

```
draw_network(n_x=2, n_h=4, n_y=1)
```



**Mathematically**, for one example  $x^{(i)}$ :

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]} \quad (1)$$

$$a^{[1](i)} = \tanh(z^{[1](i)}) \quad (2)$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]} \quad (3)$$

$$\hat{y}^{(i)} = a^{[2](i)} = \sigma(z^{[2](i)}) \quad (4)$$

$$y_{prediction}^{(i)} = \begin{cases} 1 & \text{if } a^{[2](i)} > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

Given the predictions on all the examples, you can compute the cost  $J$ :

$$J = -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \log(a^{[2](i)}) + (1 - y^{(i)}) \log(1 - a^{[2](i)}) \right) \quad (6)$$

**Reminder:** the general method for building a neural network is:

1. Define the network structure (number of input units, hidden units, and so on).
2. Initialize the model's parameters.
3. Loop:
  - Implement forward propagation
  - Compute the loss
  - Implement backward propagation to get the gradients
  - Update the parameters (gradient descent)

You will build a helper function for each step, then merge them into one function called `nn_model()`. Once `nn_model()` has learned the right parameters, you can make predictions on new data.

## ✓ 4.1 - Defining the neural network structure

### Exercise 2 - layer\_sizes

Define three variables:

- `n_x`: the size of the input layer
- `n_h`: the size of the hidden layer (**set this to 4**, `n_h = 4`, for this exercise only)
- `n_y`: the size of the output layer

**Hint:** Use the shapes of `X` and `Y` to find `n_x` and `n_y`.

```
def layer_sizes(X, Y):
    """
    Arguments:
    X -- input dataset of shape (input size, number of examples)
    Y -- labels of shape (output size, number of examples)

    Returns:
    n_x -- the size of the input layer
    n_h -- the size of the hidden layer
    n_y -- the size of the output layer
    """
    # (~ 3 lines of code)
    # YOUR CODE STARTS HERE
    n_x = X.shape[0]
    n_h = 4
    n_y = Y.shape[0]
    # YOUR CODE ENDS HERE
    return (n_x, n_h, n_y)
```

```
t_X, t_Y = np.random.randn(5, 3), np.random.randn(2, 3)
(n_x, n_h, n_y) = layer_sizes(t_X, t_Y)
print("The size of the input layer is: n_x = " + str(n_x))
print("The size of the hidden layer is: n_h = " + str(n_h))
print("The size of the output layer is: n_y = " + str(n_y))
```

```
layer_sizes_test(layer_sizes)
```

```
The size of the input layer is: n_x = 5
The size of the hidden layer is: n_h = 4
The size of the output layer is: n_y = 2
All tests passed for layer_sizes!
```

## Expected output

```
The size of the input layer is: n_x = 5
The size of the hidden layer is: n_h = 4
The size of the output layer is: n_y = 2
All tests passed for layer_sizes!
```

## ✓ 4.2 - Initialize the model's parameters

### Exercise 3 - initialize\_parameters

Implement the function `initialize_parameters()`.

#### Instructions:

- Make sure your parameters have the right sizes. Look at the network figure above if needed.
- Initialize the weight matrices with random values.
  - Use `np.random.randn(a, b) * 0.01` to create a random matrix of shape (a, b).
- Initialize the bias vectors as zeros.
  - Use `np.zeros((a, b))` to create a matrix of zeros of shape (a, b).

```
def initialize_parameters(n_x, n_h, n_y):
    """
    Arguments:
    n_x -- size of the input layer
    n_h -- size of the hidden layer
    n_y -- size of the output layer

    Returns:
    params -- python dictionary containing your parameters:
                W1 -- weight matrix of shape (n_h, n_x)
                b1 -- bias vector of shape (n_h, 1)
                W2 -- weight matrix of shape (n_y, n_h)
                b2 -- bias vector of shape (n_y, 1)
    """
    # (≈ 4 lines of code)
    # YOUR CODE STARTS HERE
    W1 = np.random.randn(n_h, n_x) * 0.01
    b1 = np.zeros((n_h, 1))
    W2 = np.random.randn(n_y, n_h) * 0.01
    b2 = np.zeros((n_y, 1))
```

```
# YOUR CODE ENDS HERE
```

```
parameters = {"W1": W1,
              "b1": b1,
              "W2": W2,
              "b2": b2}

return parameters
```

```
np.random.seed(2)
parameters = initialize_parameters(2, 4, 1)

print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))
print("b2 = " + str(parameters["b2"]))

initialize_parameters_test(initialize_parameters)
```

```
W1 = [[-0.00416758 -0.00056267]
      [-0.02136196  0.01640271]
      [-0.01793436 -0.00841747]
      [ 0.00502881 -0.01245288]]
b1 = [[0.]
      [0.]
      [0.]
      [0.]]
W2 = [[-0.01057952 -0.00909008  0.00551454  0.02292208]]
b2 = [[0.]]
All tests passed for initialize_parameters!
```

### Expected output

```
W1 = [[-0.00416758 -0.00056267]
      [-0.02136196  0.01640271]
      [-0.01793436 -0.00841747]
      [ 0.00502881 -0.01245288]]
b1 = [[0.]
      [0.]
      [0.]
      [0.]]
W2 = [[-0.01057952 -0.00909008  0.00551454  0.02292208]]
b2 = [[0.]]
All tests passed for initialize_parameters!
```

## ✓ 4.3 - Forward propagation

### Exercise 4 - forward\_propagation

Implement `forward_propagation()` using these vectorized equations:

$$Z^{[1]} = W^{[1]}X + b^{[1]} \quad (1)$$

$$A^{[1]} = \tanh(Z^{[1]}) \quad (2)$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} \quad (3)$$

$$\hat{Y} = A^{[2]} = \sigma(Z^{[2]}) \quad (4)$$

### Instructions:

- Use the function `sigmoid()`. It is defined in the lab utilities cell.
- Use the function `np.tanh()`. It is part of numpy.
- Steps:
  1. Get each parameter from the dictionary `parameters` (the output of `initialize_parameters()`) using `parameters[".."]`.
  2. Compute  $Z^{[1]}$ ,  $A^{[1]}$ ,  $Z^{[2]}$  and  $A^{[2]}$  (the vector of predictions for all examples in the training set).
- The values needed for backpropagation are stored in `cache`, which is passed to the backpropagation function.

```
def forward_propagation(X, parameters):
    """
    Argument:
    X -- input data of size (n_x, m)
    parameters -- python dictionary containing your parameters (output of initialize_para

    Returns:
    A2 -- the sigmoid output of the second activation
    cache -- a dictionary containing "Z1", "A1", "Z2" and "A2"
    """

    # Retrieve each parameter from the dictionary "parameters"
    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]
    # YOUR CODE ENDS HERE

    # Implement forward propagation to calculate A2 (probabilities)
    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    Z1 = np.dot(W1, X) + b1
    A1 = np.tanh(Z1)
    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2)
    # YOUR CODE ENDS HERE

    assert A2.shape == (1, X.shape[1])

    cache = {"Z1": Z1,
             "A1": A1,
             "Z2": Z2,
```

```
"A2": A2}
return A2, cache
```

```
A2, cache = forward_propagation(np.array(_REF["fp_X"]), {k: np.array(v) for k, v in _REF[
print("A2 = " + str(A2))
```

```
forward_propagation_test(forward_propagation)
```

```
A2 = [[0.23138802 0.6377847 0.42367349]]
All tests passed for forward_propagation!
```

## Expected output

```
A2 = [[0.23138802 0.6377847 0.42367349]]
All tests passed for forward_propagation!
```

## ✓ 4.4 - Compute the Cost

Now that you have computed  $A^{[2]}$  (the Python variable `A2`), which contains  $a^{[2](i)}$  for every example, you can compute the cost:

$$J = -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \log(a^{[2](i)}) + (1 - y^{(i)}) \log(1 - a^{[2](i)}) \right) \quad (7)$$

### Exercise 5 - compute\_cost

Implement `compute_cost()` to compute the value of the cost  $J$ .

#### Instructions:

- There are many ways to implement the cross-entropy loss. Here is one way to compute the part  $-\sum_{i=1}^m y^{(i)} \log(a^{[2](i)})$  without a for loop:

```
logprobs = np.multiply(np.log(A2), Y)
cost = - np.sum(logprobs)
```

- Use that idea to build the whole cost function.

#### Notes:

- You can use `np.multiply()` followed by `np.sum()`, or use `np.dot()` directly.
- `np.multiply` + `np.sum` gives a `float`; `np.dot` gives a 2D numpy array.
- `np.squeeze()` removes extra dimensions, and `float()` turns the result into a Python float.

```
def compute_cost(A2, Y):
    """
    Computes the cross-entropy cost given in equation (7)
```

```

Arguments:
A2 -- the sigmoid output of the second activation, of shape (1, number of examples)
Y -- "true" labels vector of shape (1, number of examples)

Returns:
cost -- cross-entropy cost given equation (7)
"""
m = Y.shape[1] # number of examples

# Compute the cross-entropy cost
# (~ 2 lines of code)
# YOUR CODE STARTS HERE
logprobs = np.multiply(np.log(A2), Y) + np.multiply(np.log(1 - A2), 1 - Y)
cost = - np.sum(logprobs) / m
# YOUR CODE ENDS HERE

cost = float(np.squeeze(cost)) # makes sure cost is a Python float, e.g. turns [[17]]
return cost

```

```

A2, t_Y = np.array(_REF["cost_A2"]), np.array(_REF["cost_Y"])
print("cost = " + str(compute_cost(A2, t_Y)))

compute_cost_test(compute_cost)

cost = 0.9385297615620027
All tests passed for compute_cost!

```

### Expected output

```

cost = 0.9385297615546655
All tests passed for compute_cost!

```

## ✓ 4.5 - Implement Backpropagation

Using the cache computed during forward propagation, you can now implement backward propagation.

### Exercise 6 - backward\_propagation

Implement the function `backward_propagation()`.

**Instructions:** Backpropagation is usually the hardest (most mathematical) part of deep learning. Because you are writing a **vectorized** implementation (all  $m$  examples at once), use these six equations:

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ^{[2](i)} \quad (\text{sum over columns: } \text{np.sum}(dZ2, \text{axis}=1, \text{keepdims}=\text{True}))$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g^{[1]'}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ^{[1](i)} \quad (\text{np.sum}(dZ1, \text{axis}=1, \text{keepdims}=\text{True}))$$

- \* means element-wise multiplication.
- Notation used in the code:  $\text{dw1} = \frac{\partial J}{\partial W^{[1]}}$ ,  $\text{db1} = \frac{\partial J}{\partial b^{[1]}}$ ,  $\text{dw2} = \frac{\partial J}{\partial W^{[2]}}$ ,  $\text{db2} = \frac{\partial J}{\partial b^{[2]}}$ .

**Tip:** To compute  $\text{dZ1}$  you need  $g^{[1]'}(Z^{[1]})$ . Because  $g^{[1]}$  is `tanh`, if  $a = g^{[1]}(z)$  then  $g^{[1]'}(z) = 1 - a^2$ . So you can compute  $g^{[1]'}(Z^{[1]})$  as `(1 - np.power(A1, 2))`.

```
def backward_propagation(parameters, cache, X, Y):
    """
    Implements backward propagation using the six equations above.

    Arguments:
    parameters -- python dictionary containing our parameters
    cache -- a dictionary containing "Z1", "A1", "Z2" and "A2"
    X -- input data of shape (2, number of examples)
    Y -- "true" labels vector of shape (1, number of examples)

    Returns:
    grads -- python dictionary containing your gradients with respect to the different pa
    """
    m = X.shape[1]

    # First, retrieve W1 and W2 from the dictionary "parameters".
    # Then, retrieve A1 and A2 from the dictionary "cache".
    # Finally, compute dw1, db1, dw2, db2.
    # (~ 10 lines of code)
    # YOUR CODE STARTS HERE
    W1 = parameters["W1"]
    W2 = parameters["W2"]

    A1 = cache["A1"]
    A2 = cache["A2"]

    dZ2 = A2 - Y
    dW2 = np.dot(dZ2, A1.T) / m
    db2 = np.sum(dZ2, axis=1, keepdims=True) / m
    dZ1 = np.dot(W2.T, dZ2) * (1 - np.power(A1, 2))
    dW1 = np.dot(dZ1, X.T) / m
    db1 = np.sum(dZ1, axis=1, keepdims=True) / m
    # YOUR CODE ENDS HERE

    grads = {"dw1": dW1,
             "db1": db1,
             "dw2": dW2,
```

```
"db2": db2}
return grads
```

```
t_params = {k: np.array(v) for k, v in _REF["fp_params"].items()}
t_cache = {k: np.array(v) for k, v in _REF["fp_cache"].items()}
t_X, t_Y = np.array(_REF["fp_X"]), np.array(_REF["bp_Y"])

grads = backward_propagation(t_params, t_cache, t_X, t_Y)
print("dw1 = " + str(grads["dw1"]))
print("db1 = " + str(grads["db1"]))
print("dw2 = " + str(grads["dw2"]))
print("db2 = " + str(grads["db2"]))

backward_propagation_test(backward_propagation)

dw1 = [[-0.00361927  0.00698779]
 [-0.1957508   0.32905277]
 [ 0.00194825 -0.00385053]
 [-0.1519739   0.71171121]]
db1 = [[ 0.00582127]
 [-0.18509617]
 [-0.00252124]
 [-0.23816571]]
dw2 = [[-5.53879998e-01  5.09924649e-04 -6.59220427e-01 -2.60571855e-03]]
db2 = [[-0.23571793]]
All tests passed for backward_propagation!
```

## Expected output

```
dw1 = [[-0.00361927  0.00698779]
 [-0.1957508   0.32905277]
 [ 0.00194825 -0.00385053]
 [-0.1519739   0.71171121]]
db1 = [[ 0.00582127]
 [-0.18509617]
 [-0.00252124]
 [-0.23816571]]
dw2 = [[-5.53879998e-01  5.09924665e-04 -6.59220427e-01 -2.60571855e-03]]
db2 = [[-0.23571793]]
All tests passed for backward_propagation!
```

## ✓ 4.6 - Update Parameters

### Exercise 7 - update\_parameters

Implement the update rule using gradient descent. Use (`dw1`, `db1`, `dw2`, `db2`) to update (`w1`, `b1`, `w2`, `b2`).

**General gradient descent rule:**  $\theta = \theta - \alpha \frac{\partial J}{\partial \theta}$ , where  $\alpha$  is the learning rate and  $\theta$  is a parameter.

A good learning rate makes the cost go down steadily; a learning rate that is too large makes it jump around or diverge. You will see this yourself in Section 7.

### Hint

- Use `copy.deepcopy(...)` when copying lists or dictionaries that are passed to functions. This stops the function from changing its input (the tests check this).

```
def update_parameters(parameters, grads, learning_rate=1.2):
    """
    Updates parameters using the gradient descent update rule given above

    Arguments:
    parameters -- python dictionary containing your parameters
    grads -- python dictionary containing your gradients
    learning_rate -- the step size alpha

    Returns:
    parameters -- python dictionary containing your updated parameters
    """
    # Retrieve a copy of each parameter from the dictionary "parameters". Use copy.deepcopy
    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    W1 = copy.deepcopy(parameters["W1"])
    b1 = copy.deepcopy(parameters["b1"])
    W2 = copy.deepcopy(parameters["W2"])
    b2 = copy.deepcopy(parameters["b2"])
    # YOUR CODE ENDS HERE

    # Retrieve each gradient from the dictionary "grads"
    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    dW1 = grads["dW1"]
    db1 = grads["db1"]
    dW2 = grads["dW2"]
    db2 = grads["db2"]
    # YOUR CODE ENDS HERE

    # Update rule for each parameter
    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    W1 = W1 - learning_rate * dW1
    b1 = b1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    b2 = b2 - learning_rate * db2

    # YOUR CODE ENDS HERE

    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2}
    return parameters
```

```
t_params = {k: np.array(v) for k, v in _REF["up_params"].items()}
```

```
t_grads = {k: np.array(v) for k, v in _REF["up_grads"].items()}
parameters = update_parameters(t_params, t_grads)

print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))
print("b2 = " + str(parameters["b2"]))

update_parameters_test(update_parameters)
```

```
W1 = [[-0.12562988 -0.03206649]
      [-0.75496522 -1.32486213]
      [-4.66188903  2.54102249]
      [ 1.68342209 -2.50367784]]
b1 = [[ 3.05100886]
      [-2.30728119]
      [ 1.77603444]
      [ 0.81903017]]
W2 = [[-0.97692105  1.94167339  1.88935209 -0.89954589]]
b2 = [[-1.05559117]]
All tests passed for update_parameters!
```

### Expected output

```
W1 = [[-0.12562988 -0.03206649]
      [-0.75496522 -1.32486213]
      [-4.66188903  2.54102249]
      [ 1.68342208 -2.50367784]]
b1 = [[ 3.05100886]
      [-2.30728119]
      [ 1.77603444]
      [ 0.81903017]]
W2 = [[-0.97692105  1.9416734  1.88935209 -0.89954589]]
b2 = [[-1.05559117]]
All tests passed for update_parameters!
```

## ✓ 4.7 - Integration

### Exercise 8 - nn\_model

Build your neural network model in `nn_model()`.

**Instructions:** The model must call the functions you wrote above, in the right order: initialize once, then in each iteration do forward propagation → cost → backpropagation → update. Pass `learning_rate` on to `update_parameters()`.

```
def nn_model(X, Y, n_h, num_iterations=10000, learning_rate=1.2, print_cost=False):
    """
    Arguments:
    X -- dataset of shape (2, number of examples)
    Y -- labels of shape (1, number of examples)
```

```

n_h -- size of the hidden layer
num_iterations -- number of iterations in the gradient descent loop
learning_rate -- learning rate of gradient descent
print_cost -- if True, print the cost every 1000 iterations

Returns:
parameters -- parameters learnt by the model. They can then be used to predict.
"""

np.random.seed(3)
n_x = layer_sizes(X, Y)[0]
n_y = layer_sizes(X, Y)[2]

# Initialize parameters
# (~ 1 line of code)
# YOUR CODE STARTS HERE
parameters = initialize_parameters(n_x, n_h, n_y)
# YOUR CODE ENDS HERE

# Loop (gradient descent)
for i in range(0, num_iterations):

    # (~ 4 lines of code)
    # YOUR CODE STARTS HERE
    # Forward propagation. Inputs: "X, parameters". Outputs: "A2, cache".
    A2, cache = forward_propagation(X, parameters)
    # Cost function. Inputs: "A2, Y". Outputs: "cost".
    cost = compute_cost(A2, Y)
    # Backpropagation. Inputs: "parameters, cache, X, Y". Outputs: "grads".
    grads = backward_propagation(parameters, cache, X, Y)
    # Gradient descent parameter update. Inputs: "parameters, grads, learning_rate". Outputs: "parameters".
    parameters = update_parameters(parameters, grads, learning_rate)
    # YOUR CODE ENDS HERE

    # Print the cost every 1000 iterations
    if print_cost and i % 1000 == 0:
        print("Cost after iteration %i: %f" % (i, cost))

return parameters

```

```
nn_model_test(nn_model)
```

All tests passed for nn\_model!

## Expected output

All tests passed for nn\_model!

## ✓ 5 - Test the Model

### 5.1 - Predict

## Exercise 9 - predict

Build `predict()`, which uses your model to make predictions. Use forward propagation to get the probabilities.

**Reminder:**  $y_{\text{prediction}} = \mathbb{1}\{\text{activation} > 0.5\} = \begin{cases} 1 & \text{if } \text{activation} > 0.5 \\ 0 & \text{otherwise} \end{cases}$

For example, to set the entries of a matrix `X` to 0 and 1 based on a threshold, you would write

`X_new = (X > threshold)`.

```
def predict(parameters, X):
    """
    Using the learned parameters, predicts a class for each example in X

    Arguments:
    parameters -- python dictionary containing your parameters
    X -- input data of size (n_x, m)

    Returns
    predictions -- vector of predictions of our model (red: 0 / blue: 1)
    """
    # Computes probabilities using forward propagation, and classifies to 0/1 using 0.5 as
    # ( $\approx$  2 lines of code)
    # YOUR CODE STARTS HERE
    A2, cache = forward_propagation(X, parameters)
    predictions = (A2 > 0.5)
    # YOUR CODE ENDS HERE
    return predictions
```

```
t_params = {k: np.array(v) for k, v in _REF["pr_params"].items()}
predictions = predict(t_params, np.array(_REF["pr_X"]))
print("Predictions: " + str(predictions))

predict_test(predict)
```

```
Predictions: [[ True  True False False False  True]]
All tests passed for predict!
```

### Expected output

```
Predictions: [[ True  True False False False  True]]
All tests passed for predict!
```

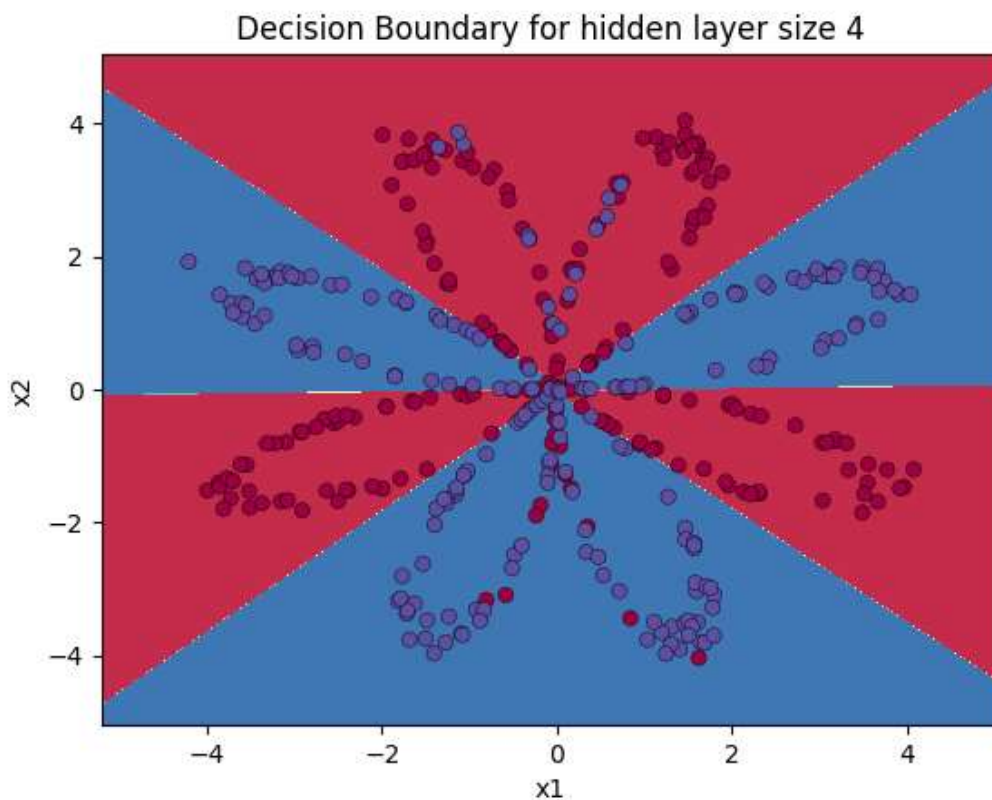
## ✓ 5.2 - Test the Model on the Planar Dataset

Now run the model and see how it does on the flower dataset. The code below trains a network with a single hidden layer of  $n_h = 4$  hidden units.

```
# Build a model with an n_h-dimensional hidden layer
parameters = nn_model(X, Y, n_h=4, num_iterations=10000, print_cost=True)

# Plot the decision boundary
plot_decision_boundary(lambda x: predict(parameters, x.T), X, Y)
plt.title("Decision Boundary for hidden layer size " + str(4))
plt.show()
```

```
Cost after iteration 0: 0.693162
Cost after iteration 1000: 0.258625
Cost after iteration 2000: 0.239334
Cost after iteration 3000: 0.230802
Cost after iteration 4000: 0.225528
Cost after iteration 5000: 0.221845
Cost after iteration 6000: 0.219094
Cost after iteration 7000: 0.220609
Cost after iteration 8000: 0.219395
Cost after iteration 9000: 0.218481
```



```
# Print accuracy
predictions = predict(parameters, X)
print('Accuracy: %d%%' % (np.mean(predictions == Y) * 100))
```

Accuracy: 90%

**Expected output:** Accuracy: 90%

The accuracy is much higher than logistic regression. The model has learned the shape of the flower's petals! Unlike logistic regression, a neural network can learn highly non-linear decision boundaries.

## 6 - Tuning the hidden layer size

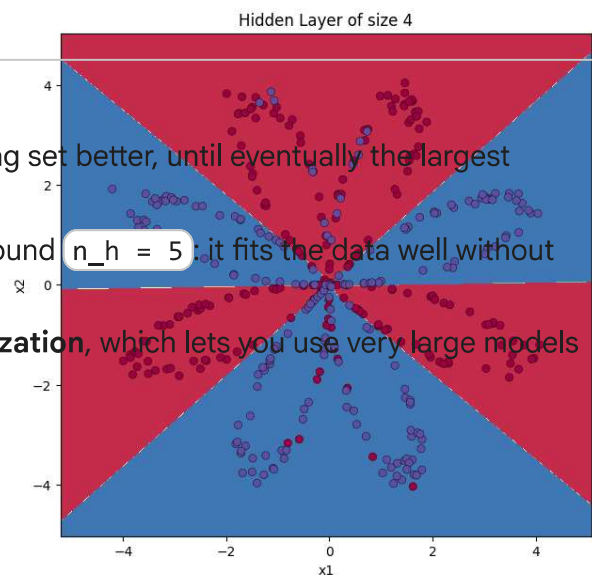
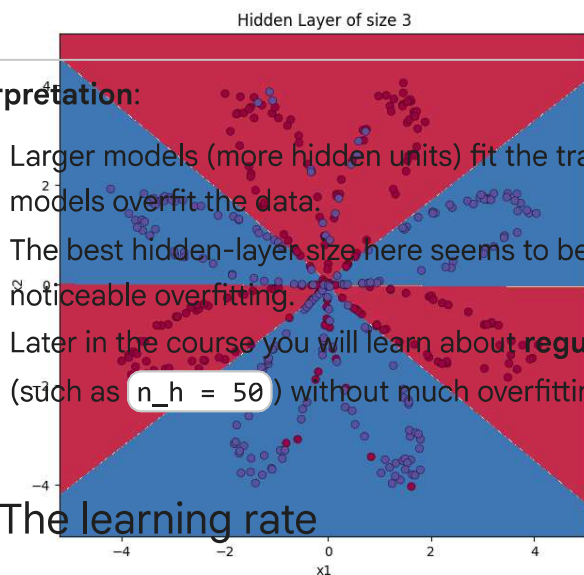
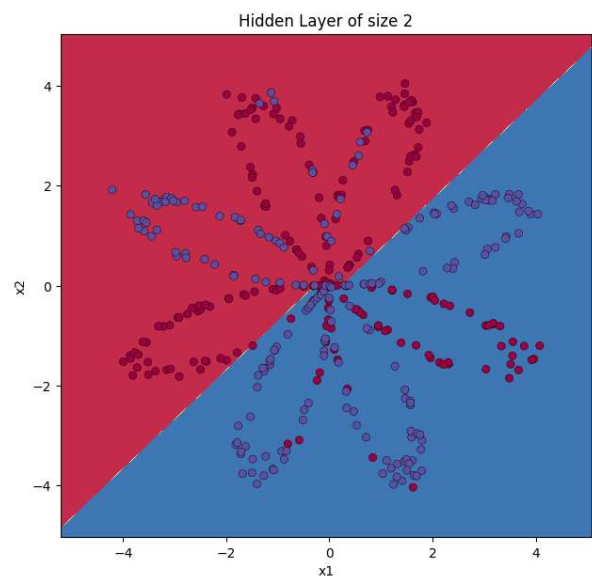
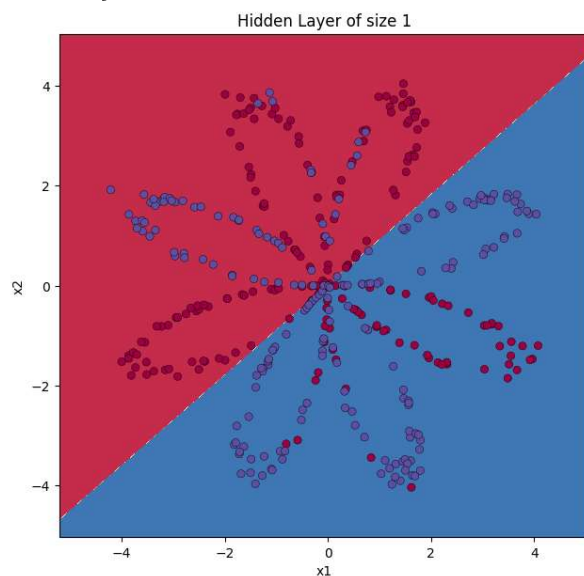
Run the code below (it can take a minute or two). Look at how the model behaves with different hidden-layer sizes.

```
# This may take about 1-2 minutes to run
plt.figure(figsize=(16, 32))
hidden_layer_sizes = [1, 2, 3, 4, 5, 20, 50]

for i, n_h in enumerate(hidden_layer_sizes):
    plt.subplot(4, 2, i + 1)
    plt.title('Hidden Layer of size %d' % n_h)
    parameters = nn_model(X, Y, n_h, num_iterations=5000)
    plot_decision_boundary(lambda x: predict(parameters, x.T), X, Y)
    predictions = predict(parameters, X)
    accuracy = np.mean(predictions == Y) * 100
    print("Accuracy for {} hidden units: {:.2f} %".format(n_h, accuracy))
plt.show()
```



Accuracy for 1 hidden units: 67.50 %  
 Accuracy for 2 hidden units: 67.25 %  
 Accuracy for 3 hidden units: 90.75 %  
 Accuracy for 4 hidden units: 90.50 %  
 Accuracy for 5 hidden units: 91.25 %  
 Accuracy for 20 hidden units: 90.75 %  
 Accuracy for 50 hidden units: 90.25 %



### Interpretation:

- Larger models (more hidden units) fit the training set better, until eventually the largest models overfit the data.
- The best hidden-layer size here seems to be around `n_h = 5`: it fits the data well without noticeable overfitting.
- Later in the course you will learn about **regularization**, which lets you use very large models (such as `n_h = 50`) without much overfitting.

## 7 - The learning rate

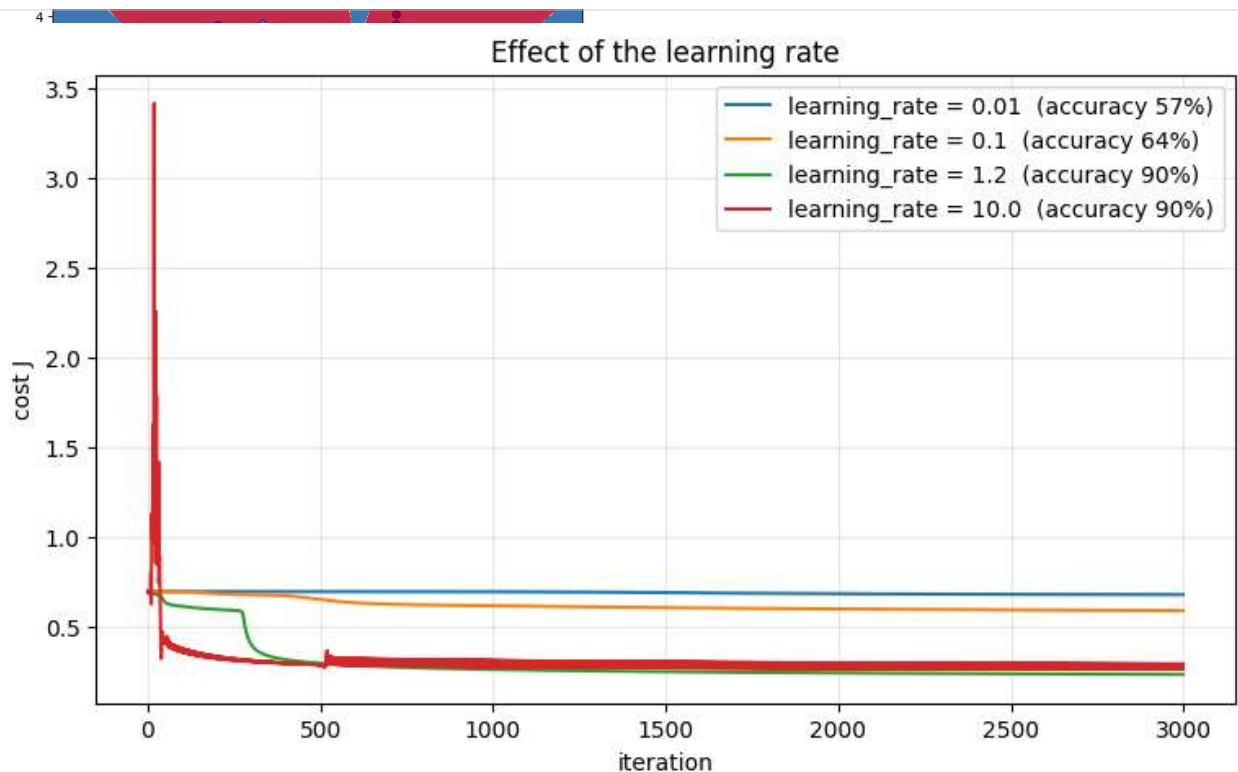
The code below trains the same 4-unit network with different learning rates and plots the cost after every iteration. It uses only the functions you wrote. Run it and compare the curves.

```
learning_rates = [0.01, 0.1, 1.2, 10.0]
plt.figure(figsize=(9, 5))
for lr in learning_rates:
    np.random.seed(3)
    params = initialize_parameters(2, 4, 1)
    costs = []
    for i in range(3000):
        A2, cache = forward_propagation(X, params)
        costs.append(compute_cost(A2, Y))
        grads = backward_propagation(params, cache, X, Y)
        params = update_parameters(params, grads, learning_rate=lr)
```

```

acc = np.mean(predict(params, X) == Y) * 100
plt.plot(costs, label=f"learning_rate = {lr} (accuracy {acc:.0f}%)")
plt.xlabel("iteration"); plt.ylabel("cost J"); plt.title("Effect of the learning rate")
plt.legend(); plt.grid(alpha=0.3); plt.show()

```



**Things to try** (optional): replace `np.tanh` with a sigmoid or a ReLU in `forward_propagation` (remember to change the derivative in `backward_propagation` too). What happens?

## 8 - Performance on other datasets

Now try your network on other datasets. Change `dataset` in the cell below to one of `"noisy_circles"`, `"noisy_moons"`, `"blobs"` or `"gaussian_quantiles"`, then run the next two cells.

```

# Datasets
noisy_circles, noisy_moons, blobs, gaussian_quantiles = load_extra_datasets()

datasets = {"noisy_circles": noisy_circles,
            "noisy_moons": noisy_moons,
            "blobs": blobs,
            "gaussian_quantiles": gaussian_quantiles}

### CHOOSE YOUR DATASET HERE ###
dataset = "noisy_moons"
#####

X2, Y2 = datasets[dataset]

```

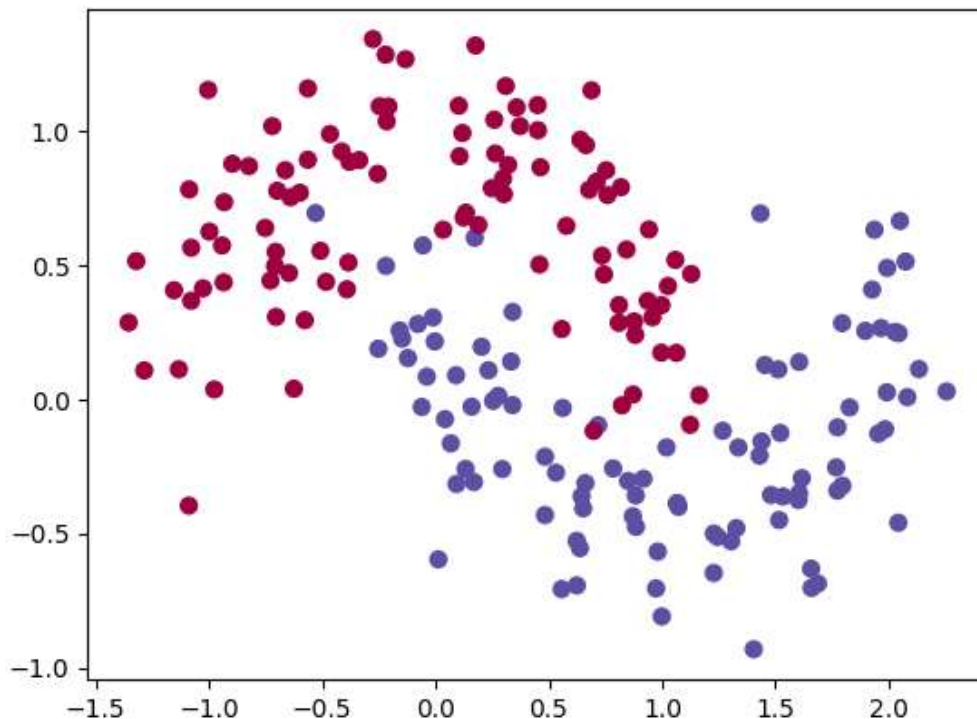
```

X2, Y2 = X2.T, Y2.reshape(1, Y2.shape[0])

# make blobs binary
if dataset == "blobs":
    Y2 = Y2 % 2

# Visualize the data
plt.scatter(X2[0, :], X2[1, :], c=Y2.ravel(), s=40, cmap=plt.cm.Spectral);

```



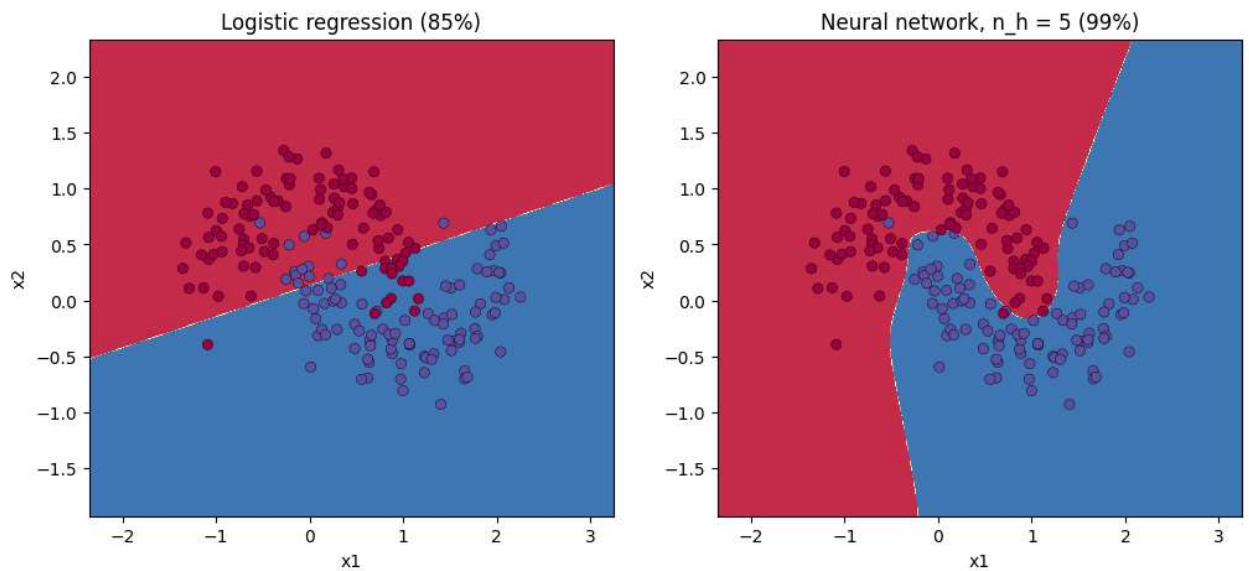
```

# Train your network on the chosen dataset and compare with logistic regression
n_h = 5
params2 = nn_model(X2, Y2, n_h=n_h, num_iterations=5000)
nn_acc = np.mean(predict(params2, X2) == Y2) * 100

clf2 = sklearn.linear_model.LogisticRegressionCV().fit(X2.T, Y2.ravel())
lr_acc = np.mean(clf2.predict(X2.T) == Y2.ravel()) * 100

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1); plt.title(f"Logistic regression ({lr_acc:.0f}%)")
plot_decision_boundary(lambda x: clf2.predict(x), X2, Y2)
plt.subplot(1, 2, 2); plt.title(f"Neural network, n_h = {n_h} ({nn_acc:.0f}%)")
plot_decision_boundary(lambda x: predict(params2, x.T), X2, Y2)
plt.show()

```



```
# Datasets
noisy_circles, noisy_moons, blobs, gaussian_quantiles = load_extra_datasets()

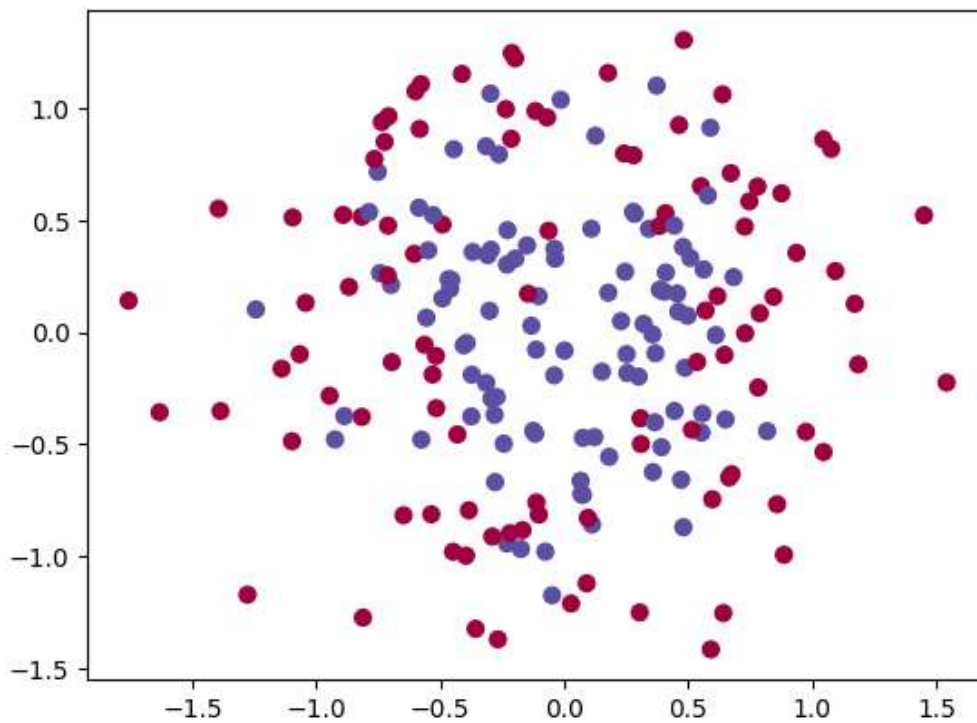
datasets = {"noisy_circles": noisy_circles,
            "noisy_moons": noisy_moons,
            "blobs": blobs,
            "gaussian_quantiles": gaussian_quantiles}

### CHOOSE YOUR DATASET HERE ###
dataset = "noisy_circles"
#####

X2, Y2 = datasets[dataset]
X2, Y2 = X2.T, Y2.reshape(1, Y2.shape[0])

# make blobs binary
if dataset == "blobs":
    Y2 = Y2 % 2

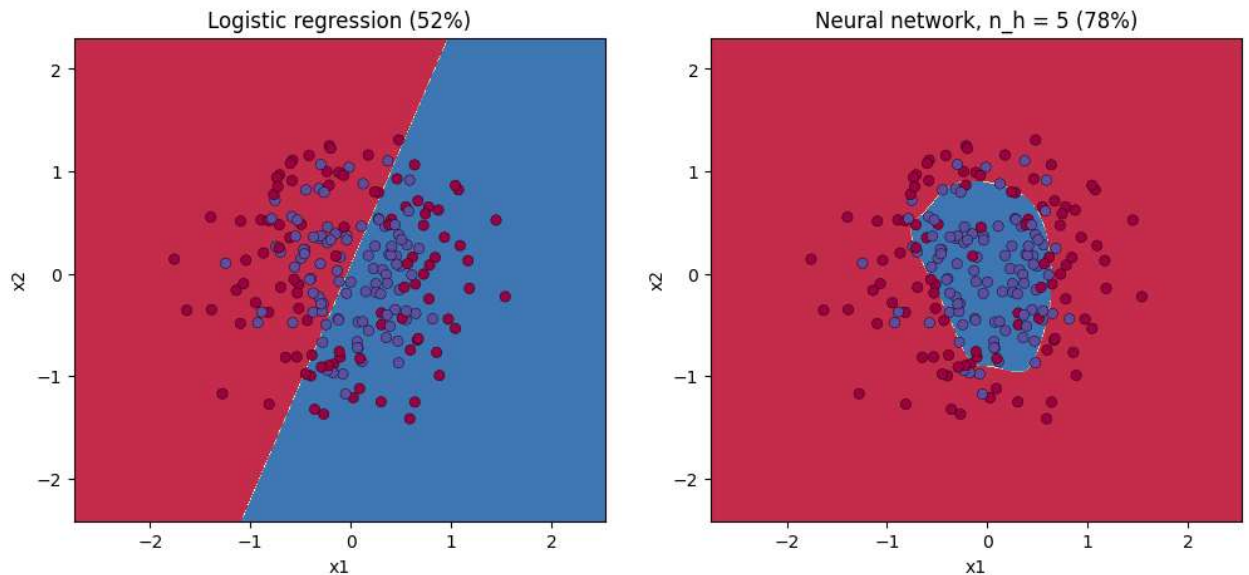
# Visualize the data
plt.scatter(X2[0, :], X2[1, :], c=Y2.ravel(), s=40, cmap=plt.cm.Spectral);
```



```
# Train your network on the chosen dataset and compare with logistic regression
n_h = 5
params2 = nn_model(X2, Y2, n_h=n_h, num_iterations=5000)
nn_acc = np.mean(predict(params2, X2) == Y2) * 100

clf2 = sklearn.linear_model.LogisticRegressionCV().fit(X2.T, Y2.ravel())
lr_acc = np.mean(clf2.predict(X2.T) == Y2.ravel()) * 100

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1); plt.title(f"Logistic regression ({lr_acc:.0f}%)")
plot_decision_boundary(lambda x: clf2.predict(x), X2, Y2)
plt.subplot(1, 2, 2); plt.title(f"Neural network, n_h = {n_h} ({nn_acc:.0f}%)")
plot_decision_boundary(lambda x: predict(params2, x.T), X2, Y2)
plt.show()
```



```
# Datasets
noisy_circles, noisy_moons, blobs, gaussian_quantiles = load_extra_datasets()

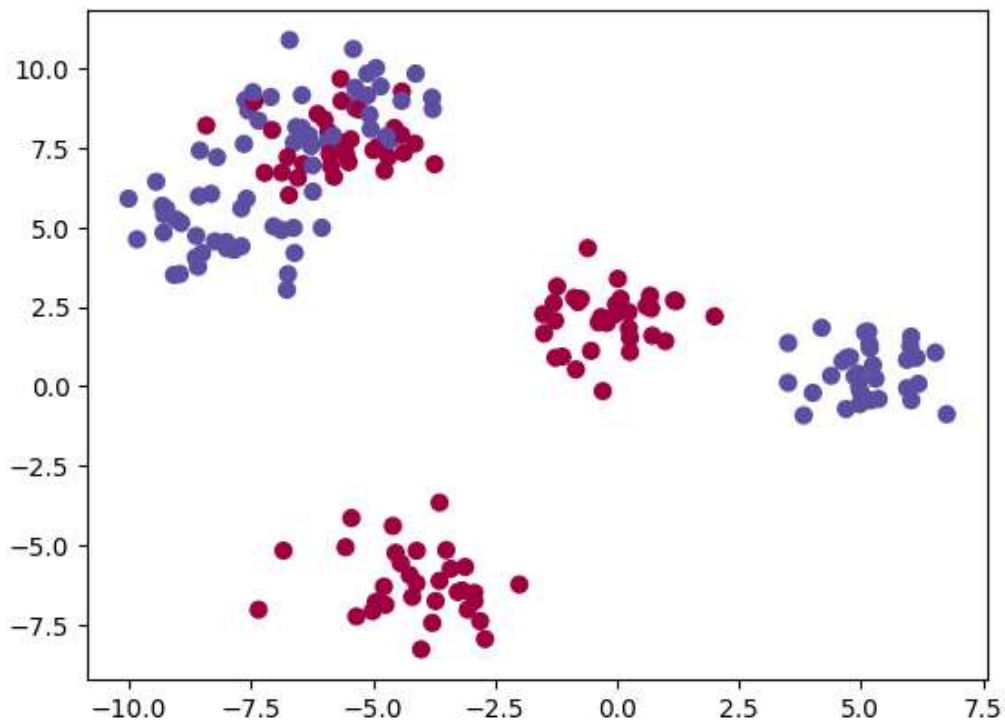
datasets = {"noisy_circles": noisy_circles,
            "noisy_moons": noisy_moons,
            "blobs": blobs,
            "gaussian_quantiles": gaussian_quantiles}

### CHOOSE YOUR DATASET HERE ###
dataset = "blobs"
#####

X2, Y2 = datasets[dataset]
X2, Y2 = X2.T, Y2.reshape(1, Y2.shape[0])

# make blobs binary
if dataset == "blobs":
    Y2 = Y2 % 2

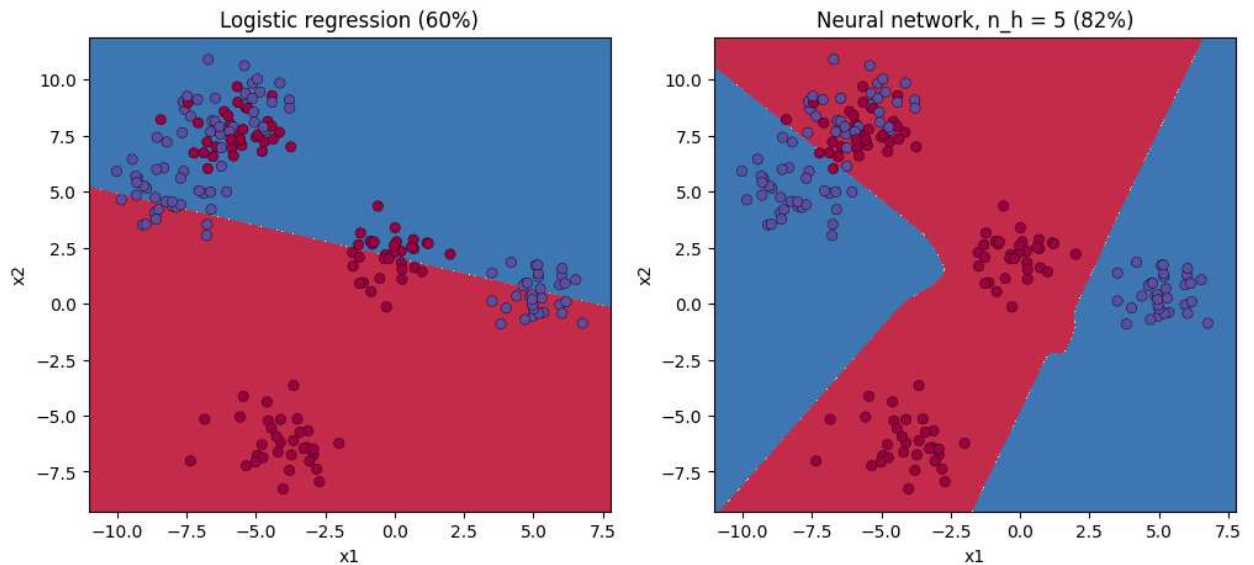
# Visualize the data
plt.scatter(X2[0, :], X2[1, :], c=Y2.ravel(), s=40, cmap=plt.cm.Spectral);
```



```
# Train your network on the chosen dataset and compare with logistic regression
n_h = 5
params2 = nn_model(X2, Y2, n_h=n_h, num_iterations=5000)
nn_acc = np.mean(predict(params2, X2) == Y2) * 100

clf2 = sklearn.linear_model.LogisticRegressionCV().fit(X2.T, Y2.ravel())
lr_acc = np.mean(clf2.predict(X2.T) == Y2.ravel()) * 100

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1); plt.title(f"Logistic regression ({lr_acc:.0f}%)")
plot_decision_boundary(lambda x: clf2.predict(x), X2, Y2)
plt.subplot(1, 2, 2); plt.title(f"Neural network, n_h = {n_h} ({nn_acc:.0f}%)")
plot_decision_boundary(lambda x: predict(params2, x.T), X2, Y2)
plt.show()
```



```
# Datasets
noisy_circles, noisy_moons, blobs, gaussian_quantiles = load_extra_datasets()

datasets = {"noisy_circles": noisy_circles,
            "noisy_moons": noisy_moons,
            "blobs": blobs,
            "gaussian_quantiles": gaussian_quantiles}

### CHOOSE YOUR DATASET HERE ###
dataset = "gaussian_quantiles"
#####

X2, Y2 = datasets[dataset]
X2, Y2 = X2.T, Y2.reshape(1, Y2.shape[0])

# make blobs binary
if dataset == "blobs":
    Y2 = Y2 % 2

# Visualize the data
plt.scatter(X2[0, :], X2[1, :], c=Y2.ravel(), s=40, cmap=plt.cm.Spectral);
```